

Лабораторна робота №1

Тензори та матричні операції

Роман Миджин · 25 вересня 2026 р.

Мета: у цій роботі я реалізував основні операції з векторами та матрицями, зробив лінійний шар і порівняв швидкість множення на CPU та GPU.

Завдання: [lpnu-llm/lab-tensors](https://github.com/lpnu-llm/lab-tensors)

1. Додавання векторів

тут додаю відповідні числа з двох векторів. якщо довжини різні, видаю помилку.

```
In [1]: def vector_add(a, b):
        """додає два вектори поелементно"""
        if len(a) != len(b):
            raise ValueError("Вектори повинні мати однакову довжину")
        result = []
        for i in range(len(a)):
            result.append(a[i] + b[i])
        return result

def test_vector_add():
    assert vector_add([1, 2, 3], [4, 5, 6]) == [5, 7, 9]
    assert vector_add([0.5, 1.5], [0.25, -0.5]) == [0.75, 1.0]
    assert vector_add([], []) == []

    try:
        vector_add([1, 2], [3, 4, 5])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Додавання векторів різних довжин має давати помилку")

if __name__ == "__main__":
    test_vector_add()
    print("✓ vector_add")
```

✓ vector_add

2. Скалярний добуток

множу відповідні елементи векторів і складаю результати. на виході одне число.

```
In [2]: def dot(a, b):
        """рахує скалярний добуток"""
        if len(a) != len(b):
            raise ValueError("Вектори повинні мати однакову довжину")
        result = 0
        for i in range(len(a)):
            result += a[i] * b[i]
        return result

def test_dot():
    assert dot([1, 2, 3], [4, 5, 6]) == 32
    assert dot([2, -3], [4, 0.5]) == 6.5
    assert dot([], []) == 0

    assert dot([1, 0], [0, 1]) == 0

    try:
        dot([1, 2], [3])
    except ValueError:
        pass
    else:
        raise AssertionError("Добуток векторів різних неможливий")

if __name__ == "__main__":
    test_dot()
    print("✓ dot")
```

✓ dot

3. Матриця × вектор

для кожного рядка матриці рахую скалярний добуток із вектором.

```
In [3]: def matvec(mat, vec):
        """множить кожен рядок матриці на вектор"""
        if not mat:
            raise ValueError("Порожня матриця не задає кількість стовпців")
        result = []
        for row in mat:
            if len(row) != len(vec):
                raise ValueError("Довжина кожного рядка має дорівнювати довжині вектора")
            result.append(dot(row, vec))
        return result

def test_matvec():
    assert matvec(
        [[1, 2, 3],
         [4, 5, 6]],
        [10, 20, 30],
    ) == [140, 320]

    assert matvec(
        [[1, 0],
         [0, 1]],
        [1, 0],
    ) == [1, 0]
```

```

        [7, -3],
    ) == [7, -3]

    assert matvec([[2, 3]], [4, 5]) == [23]

    assert matvec([[2], [3], [4]], [10]) == [20, 30, 40]

    assert matvec([[0, 0], [0, 0]], [5, 6]) == [0, 0]
    assert matvec([], [], []) == [0.0, 0.0]

    a = [[1, 2], [3, 4]]
    x = [5, 6]
    matvec(a, x)
    assert a == [[1, 2], [3, 4]]
    assert x == [5, 6]

    try:
        matvec([
            [1, 2],
            [3, 4]],
            [1])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: довжина вектора не відповідає довжині рядків"
        )

    try:
        matvec([
            [1, 2],
            [3]],
            [1, 2])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: матриця має рядки різної довжини"
        )

    try:
        matvec([], [])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: порожня матриця не визначає кількість стовпців"
        )

    print("Усі перевірки matvec пройдено.")

if __name__ == "__main__":
    test_matvec()
    print("✓ matvec")

```

Усі перевірки matvec пройдено.
✓ matvec

4. Форма тензора

перевіряю вкладені списки і записую їхні розміри. рядки різної довжини не підходять.

```
In [4]: def shape(tensor):
        """повертає розміри тензора"""
        if not isinstance(tensor, (list, tuple)):
            return ()
        if not tensor:
            return (0,)
        inner_shape = shape(tensor[0])
        for item in tensor[1:]:
            if shape(item) != inner_shape:
                raise ValueError("Усі вкладені елементи повинні мати однакову форму")
        return (len(tensor),) + inner_shape

def test_shape():
    assert shape(7) == ()
    assert shape(3.14) == ()

    assert shape([10, 20, 30]) == (3,)
    assert shape([]) == (0,)

    assert shape(
        [[1, 2, 3],
         [4, 5, 6]],
    ) == (2, 3)
    assert shape([[1], [2], [3]]) == (3, 1)
    assert shape([[], []]) == (2, 0)

    assert shape(
        [[[1, 2], [3, 4]],
         [[5, 6], [7, 8]]],
    ) == (2, 2, 2)

    try:
        shape([[1, 2], [3]])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: вкладені списки мають різну форму"
        )

    try:
        shape([1, [2]])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: тензор змішує скаляри та списки"
```

```

    )

    print("Усі перевірки shape пройдено.")

if __name__ == "__main__":
    test_shape()
    print("✓ shape")

```

Усі перевірки shape пройдено.

✓ shape

5. Лінійний шар

спочатку множу матрицю ваг на вхідний вектор, потім додаю bias.

```

In [5]: def linear(x, weight, bias):
        """рахує один лінійний шар: Wx + b"""
        result = matvec(weight, x)
        return vector_add(result, bias)

def test_linear():

    assert linear(
        [2, 3],
        [[1, 0],
         [0, 1],
         [1, 1]],
        [10, 20, 30],
    ) == [12, 23, 35]

    assert linear(
        [2, 3, 4],
        [[1, 10, 100]],
        [5],
    ) == [437]

    assert linear(
        [7, -3],
        [[0, 0],
         [0, 0]],
        [1, -2],
    ) == [1, -2]

    try:
        linear(
            [1, 2],
            [[1, 0],
             [0, 1]],
            [10],
        )
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: довжина bias не дорівнює out_dim"

```

```

    )

    print("Усі перевірки linear пройдено.")

if __name__ == "__main__":
    test_linear()
    print("✓ linear")

```

Усі перевірки linear пройдено.

✓ linear

6. Матриця × матриця

кожне число в результаті отримую з скалярного добутку рядка першої матриці та стовпця другої.

```

In [6]: def matmul(a, b):
        """множить дві матриці"""
        a_shape = shape(a)
        b_shape = shape(b)
        if len(a_shape) != 2 or len(b_shape) != 2:
            raise ValueError("Обидва аргументи повинні бути матрицями")
        rows_a, cols_a = a_shape
        rows_b, cols_b = b_shape
        if cols_a != rows_b:
            raise ValueError("Внутрішні розміри матриць повинні збігатися")
        result = []
        for row in a:
            result_row = []
            for j in range(cols_b):
                column = [b[i][j] for i in range(rows_b)]
                result_row.append(dot(row, column))
            result.append(result_row)
        return result

def test_matmul():
    assert matmul(
        [[1, 2, 3],
         [4, 5, 6]],
        [[1, 2],
         [3, 4],
         [5, 6]],
    ) == [[22, 28], [49, 64]]

    assert matmul(
        [[1, 2],
         [3, 4],
         [5, 6]],
        [[1, 0, 1, 0],
         [0, 1, 0, 1]],
    ) == [[1, 2, 1, 2],
         [3, 4, 3, 4],
         [5, 6, 5, 6]]

```

```

assert matmul(
    [[2, -1],
     [7, 3]],
    [[1, 0],
     [0, 1]],
) == [[2, -1], [7, 3]]

try:
    matmul(
        [[1, 2, 3]],
        [[1, 2],
         [3, 4]],
    )
except ValueError:
    pass
else:
    raise AssertionError(
        "Очікується ValueError: внутрішні розміри матриць не збігаються"
    )

try:
    matmul(
        [[1, 2],
         [3]],
        [[1],
         [2]],
    )
except ValueError:
    pass
else:
    raise AssertionError(
        "Очікується ValueError: матриця a має рядки різної довжини"
    )

print("Усі перевірки matmul пройдено.")

if __name__ == "__main__":
    test_matmul()
    print("✓ matmul")

```

Усі перевірки matmul пройдено.

✓ matmul

7. Транспонування

тут рядки стають стовпцями, а стовпці рядками.

```

In [7]: def transpose(mat):
        """мінє місцями рядки і стовпці"""
        matrix_shape = shape(mat)
        if len(matrix_shape) != 2:
            raise ValueError("Аргумент повинен бути матрицею")
        rows, columns = matrix_shape
        result = []
        for j in range(columns):

```

```

        column = []
        for i in range(rows):
            column.append(mat[i][j])
        result.append(column)
    return result

def test_transpose():
    assert transpose(
        [[1, 2, 3],
         [4, 5, 6]],
    ) == [[1, 4],
          [2, 5],
          [3, 6]]

    assert transpose([[1, 2, 3]]) == [[1], [2], [3]]

    assert transpose([[1], [2], [3]]) == [[1, 2, 3]]

    assert transpose([[1, 2], [3, 4]]) == [[1, 3], [2, 4]]

    try:
        transpose([[1, 2], [3]])
    except ValueError:
        pass
    else:
        raise AssertionError(
            "Очікується ValueError: матриця має рядки різної довжини"
        )

    print("Усі перевірки transpose пройдено.")

if __name__ == "__main__":
    test_transpose()
    print("✓ transpose")

```

Усі перевірки transpose пройдено.
✓ transpose

8. Лінійний шар для батчу

застосовую однакові ваги до всіх векторів у батчі.

```

In [8]: def linear_batch(xs, weight, bias):
        """рахує лінійний шар для кількох векторів"""
        weight_transposed = transpose(weight)
        products = matmul(xs, weight_transposed)
        result = []
        for row in products:
            result.append(vector_add(row, bias))
        return result

def test_linear_batch():
    xs = [[2, 3],
           [4, 5],
           [6, 7]]

```

```

weight = [[1, 0],
           [0, 1],
           [1, 1]]
bias = [10, 20, 30]

result = linear_batch(xs, weight, bias)

assert result == [[12, 23, 35],
                  [14, 25, 39],
                  [16, 27, 43]]

assert result == [linear(x, weight, bias) for x in xs]

print("Усі перевірки linear_batch пройдено.")

if __name__ == "__main__":
    test_linear_batch()
    print("✓ linear_batch")

```

Усі перевірки linear_batch пройдено.
✓ linear_batch

9. Порівняння двох реалізацій на Python

порівнюю обробку батчу циклом і через одне множення матриць.

```

In [9]: def compare_matvec_and_matmul():
        """Порівнює час обробки батчу двома способами."""
        from time import perf_counter

        batch_size = 32
        input_dim = 64
        out_dim = 48
        repeats = 5

        xs = [
            [(sample + feature) % 10 for feature in range(input_dim)]
            for sample in range(batch_size)
        ]
        weight = [
            [(output - feature) % 7 for feature in range(input_dim)]
            for output in range(out_dim)
        ]

        weight_transposed = transpose(weight)

        by_matvec = [matvec(weight, x) for x in xs]
        by_matmul = matmul(xs, weight_transposed)
        assert by_matvec == by_matmul

        start = perf_counter()
        for _ in range(repeats):
            [matvec(weight, x) for x in xs]
        matvec_seconds = perf_counter() - start

```

```

start = perf_counter()
for _ in range(repeats):
    matmul(xs, weight_transposed)
matmul_seconds = perf_counter() - start

print(f"Цикл з matvec: {matvec_seconds * 1000:.2f} мс")
print(f"Один matmul:      {matmul_seconds * 1000:.2f} мс")
print(f"Відношення:      {matvec_seconds / matmul_seconds:.2f}x")

if __name__ == "__main__":
    compare_matvec_and_matmul()

```

Цикл з matvec: 27.45 мс
 Один matmul: 45.50 мс
 Відношення: 0.60x

10. Порівняння з PyTorch на CPU та GPU

порівнюю свій варіант на Python з PyTorch на процесорі та відеокарті.

```

In [10]: def compare_with_torch():
    """Порівнює навчальний matmul з PyTorch на CPU та GPU."""
    from time import perf_counter

    try:
        import torch
    except ImportError:
        print("PyTorch не встановлено. Встановіть його, щоб запуски цей бенчмарк.")
        return

    def synchronize(device):
        if device.type == "cuda":
            torch.cuda.synchronize(device)
        elif device.type == "mps":
            torch.mps.synchronize()

    def measure_torch(a, b, repeats):

        for _ in range(3):
            torch.matmul(a, b)
            synchronize(a.device)

        start = perf_counter()
        for _ in range(repeats):
            torch.matmul(a, b)
            synchronize(a.device)
        return (perf_counter() - start) / repeats

    def make_python_matrix(size, offset):
        return [
            [float((row + column + offset) % 7) for column in range(size)]
            for row in range(size)
        ]

    def benchmark_size(size, python_repeats, torch_repeats):

```

```

a = make_python_matrix(size, 0)
b = make_python_matrix(size, 1)

start = perf_counter()
python_result = None
for _ in range(python_repeats):
    python_result = matmul(a, b)
python_time = (perf_counter() - start) / python_repeats

a_cpu = torch.tensor(a, dtype=torch.float32)
b_cpu = torch.tensor(b, dtype=torch.float32)
cpu_time = measure_torch(a_cpu, b_cpu, torch_repeats)

expected = torch.tensor(python_result, dtype=torch.float32)
assert torch.allclose(torch.matmul(a_cpu, b_cpu), expected)

return python_time, cpu_time, a_cpu, b_cpu

if torch.cuda.is_available():
    gpu_device = torch.device("cuda")
elif hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
    gpu_device = torch.device("mps")
else:
    gpu_device = None

print(f"PyTorch {torch.__version__}")
print("\nHaw matmul i PyTorch CPU:")
print(f"{'size':>10} {'Python':>14} {'PyTorch CPU':>14} {'CPU speedup':>14}")

small = benchmark_size(16, python_repeats=20, torch_repeats=200)
large = benchmark_size(128, python_repeats=1, torch_repeats=30)

for size, result in ((16, small), (128, large)):
    python_time, cpu_time, _, _ = result
    print(
        f"{'{size}x{size}':>10} "
        f"{'python_time * 1000:>11.3f} mc "
        f"{'cpu_time * 1000:>11.3f} mc "
        f"{'python_time / cpu_time:>13.1f}x"
    )

if gpu_device is None:
    print("\nCUDA або MPS недоступні: GPU-бенчмарк пропущено.")
    return

print(f"\nPyTorch CPU i {gpu_device.type.upper()} GPU:")
print(f"{'size':>10} {'PyTorch CPU':>14} {gpu_device.type.upper() + ' GPU':>14}")

_, small_cpu_time, small_a_cpu, small_b_cpu = small
small_a_gpu = small_a_cpu.to(gpu_device)
small_b_gpu = small_b_cpu.to(gpu_device)
small_gpu_time = measure_torch(small_a_gpu, small_b_gpu, repeats=200)
print(
    f"{'16x16':>10} "
    f"{'small_cpu_time * 1000:>11.3f} mc "
    f"{'small_gpu_time * 1000:>11.3f} mc "

```

```

        f"{small_cpu_time / small_gpu_time:>13.2f}x"
    )

_, large_cpu_time, large_a_cpu, large_b_cpu = large
large_a_gpu = large_a_cpu.to(gpu_device)
large_b_gpu = large_b_cpu.to(gpu_device)
large_gpu_time = measure_torch(large_a_gpu, large_b_gpu, repeats=30)
print(
    f"'128x128':>10} "
    f"{large_cpu_time * 1000:>11.3f} мс "
    f"{large_gpu_time * 1000:>11.3f} мс "
    f"{large_cpu_time / large_gpu_time:>13.2f}x"
)

huge_size = 1024
huge_a_cpu = torch.randn(huge_size, huge_size)
huge_b_cpu = torch.randn(huge_size, huge_size)
huge_cpu_time = measure_torch(huge_a_cpu, huge_b_cpu, repeats=10)
huge_a_gpu = huge_a_cpu.to(gpu_device)
huge_b_gpu = huge_b_cpu.to(gpu_device)
huge_gpu_time = measure_torch(huge_a_gpu, huge_b_gpu, repeats=10)
print(
    f"'1024x1024':>10} "
    f"{huge_cpu_time * 1000:>11.3f} мс "
    f"{huge_gpu_time * 1000:>11.3f} мс "
    f"{huge_cpu_time / huge_gpu_time:>13.2f}x"
)

print("\nзначення можуть відрізнятися залежно від комп'ютера та його поточного

if __name__ == "__main__":
    compare_with_torch()

```

PyTorch 2.11.0+cu128

Наш matmul і PyTorch CPU:

size	Python	PyTorch CPU	CPU speedup
16x16	0.501 мс	0.003 мс	151.6x
128x128	231.821 мс	0.019 мс	11914.7x

PyTorch CPU і CUDA GPU:

size	PyTorch CPU	CUDA GPU	GPU speedup
16x16	0.003 мс	0.017 мс	0.19x
128x128	0.019 мс	0.024 мс	0.81x
1024x1024	7.756 мс	0.716 мс	10.83x

Значення можуть відрізнятися залежно від комп'ютера та його поточного навантаження.

Мої результати CPU і GPU

Я запускав бенчмарк на ноутбучі з AMD Ryzen 5 4600H і NVIDIA GeForce RTX 2060. Використовував Python 3.12.7, PyTorch 2.11.0+cu128 і CUDA 12.8.

Для порівняння взяв матриці `float32` і встановив `seed=42`. Матриці для CPU і GPU створив до початку вимірювань. Спочатку зробив 10 прогрівальних запусків, потім 5 серій вимірювань. У таблиці вказана медіана часу одного множення. Після запусків на GPU я чекав на їх завершення, а результати CPU і GPU порівняв між собою.

Розмір	CPU, мс	GPU, мс	CPU/GPU
16×16	0.0049	0.0414	0.12×
128×128	0.0322	0.0312	1.03×
1024×1024	9.4046	0.6817	13.80×
2048×2048	63.5741	2.9659	21.44×

Коли CPU/GPU більше за 1, GPU працює швидше. Тут я вимірював саме множення матриць, без часу на копіювання даних між оперативною пам'яттю та відеопам'яттю.

Висновок

Я реалізував функції з завдання та перевірів їх тестами. На маленькій матриці 16×16 CPU виявився швидшим: GPU витрачає час на запуск операції. На 128×128 час майже однаковий. Для більших матриць GPU уже має перевагу: на 2048×2048 множення вийшло приблизно у 21,44 рази швидшим. Також я реалізував лінійний шар для батчу за формулою $X @ W.T + b$.